

OSSP PRACTICAL_5

Task-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
GNU nano 7.2 praca
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define BUFFER_SIZE 1000

int main() {
    int pipefd[2];
    pid_t pid;
    int data[BUFFER_SIZE];
    struct timespec start, end;

    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child - Consumer
        close(pipefd[1]);

        int received[BUFFER_SIZE];
        ssize_t bytes_read;

        bytes_read = read(pipefd[0], received, sizeof(received));

        if (bytes_read > 0) {
            printf("Consumer (Child): Received %ld bytes\n", bytes_read);
        }
    }
}
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```
GNU nano 7.2                                                                    pra5task1.c
    data[i] = i + 1;
}

clock_gettime(CLOCK_MONOTONIC, &start);

ssize_t bytes_written = write(
    pipefd[1],
    data,
    sizeof(data)
);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

printf("Producer (Parent): Generated %d integers\n", BUFFER_SIZE);
printf("Producer: Sent %ld bytes\n", bytes_written);
printf("Communication time: %.9f seconds\n", elapsed);

if (elapsed > 0) {
    double throughput = bytes_written / elapsed;
    printf("Communication throughput: %.2f bytes/second\n",
        throughput);
}

close(pipefd[1]);

wait(NULL);

printf("Producer: Consumer process completed.\n");
}

return 0;
}

ruhi0404@Ruhitha:~/Operating_Systems$ nano pra5task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc pra5task1.c -o pra5task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./pra5task1
Producer (Parent): Generated 1000 integers
Consumer (Child): Received 4000 bytes
Producer: Sent 4000 bytes
First 5 values consumed: 1 2 3 4 5
Communication time: 0.000011100 seconds
Communication throughput: 360360360.36 bytes/second
Producer: Consumer process completed.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

- An anonymous pipe was successfully created for communication between the parent and child processes.
- The parent process acted as the producer and generated 1000 integer values.
- The child process acted as the consumer and read the data from the pipe.

- The parent and child communicated successfully without using a shared file.
- The communication time was measured using `clock_gettime()`.
- Communication throughput was calculated in bytes per second, providing a measure of communication efficiency.
- The parent used `wait()` to synchronize with the child and ensure that the consumer completed before the parent terminated.
- Thus, anonymous pipes provide a simple and efficient one-way inter-process communication (IPC) mechanism between related processes.

Task -2 Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
GNU nano 7.2                                                                    pi
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    // Create first child for ls -l
    pid1 = fork();

    if (pid1 < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid1 == 0) {
        // Child 1: execute ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls failed");
        exit(1);
    }

    // Create second child for grep
    pid2 = fork();
```

GNU nano 7.2 interface showing keyboard shortcuts:

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execut
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justif

System status bar at the bottom:

- Weather: 24°C, Mostly cloudy
- Search bar with magnifying glass icon
- Taskbar icons: Windows logo, Search, Football, Document, Browser

```
        exit(1);
    }

    // Create second child for grep
    pid2 = fork();

    if (pid2 < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid2 == 0) {
        // Child 2: execute grep .c
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep failed");
        exit(1);
    }

    // Parent closes both pipe ends
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both children
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
```

```

Communication throughput: 360360360.36 bytes/second
Producer: Consumer process completed.
ruhi0404@Ruhitha:~/Operating_Systems$ nano pra5task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc pra5task2.c -o pra5task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./pra5task2
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 docs
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 include
-rw-r--r-- 1 ruhi0404 ruhi0404 819 Aug 10 13:56 pra1task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 930 Aug 8 03:16 pra3task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 377 Aug 8 03:18 pra3task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1475 Aug 29 03:04 pra4task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 595 Aug 29 03:14 pra4task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1965 Aug 29 03:26 pra5task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1223 Aug 29 03:29 pra5task2.c
-rwxr-xr-x 1 ruhi0404 ruhi0404 16480 Aug 11 06:53 prac1task1
-rw-r--r-- 1 ruhi0404 ruhi0404 1021 Aug 11 06:53 prac1task1.c
-rwxr-xr-x 1 ruhi0404 ruhi0404 16224 Aug 11 07:03 prac2task1
-rw-r--r-- 1 ruhi0404 ruhi0404 1222 Aug 11 07:03 prac2task1.c
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:43 practical
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 scripts
-rw-r--r-- 1 ruhi0404 ruhi0404 1788 Aug 11 06:43 skill1task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 588 Aug 11 06:32 skill2task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1096 Aug 11 06:35 skill2task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 874 Aug 10 13:34 skill2w1task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 2029 Aug 10 13:38 skill2w1task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1071 Aug 18 06:33 skill4s7task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1301 Aug 18 06:41 skill4s7task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 2536 Aug 18 06:49 skill4s8task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1277 Aug 12 13:37 skills3task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1206 Aug 12 13:27 skills3week2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 852 Aug 12 13:41 skills4task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 2178 Aug 12 13:45 skills4task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1291 Aug 12 13:56 skills5task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 2190 Aug 12 14:02 skills5task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1556 Aug 12 14:10 skills6task1.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1019 Aug 12 14:14 skills6task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1652 Aug 18 08:28 skills8ttask2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 1177 Aug 10 13:01 skillw3task2.c
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 src
Pipeline execution completed.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

- A pipe was created using the pipe() system call to establish communication between two child processes.
- The first child used dup2() to redirect its standard output to the pipe and executed ls -l using exec().
- The second child redirected its standard input from the pipe using dup2() and executed grep .c using exec().

- The output produced by `ls -l` was therefore passed directly to `grep`, just like a normal shell pipeline.
- The parent process closed the unused pipe ends and used `waitpid()` to synchronize with both child processes.
- The program successfully demonstrated IPC using pipes, file-descriptor redirection using `dup2()`, process creation using `fork()`, and command execution using `exec()`.